App-Level TinyML for Smartphone Battery-Life Prediction: A Methodologically Honest Negative Result

Keno Schürger
Technische Hochschule Würzburg-Schweinfurt (THWS)
Email: keno.schuerger@study.thws.de

***Abstract*—We examine whether a TinyML model running inside an Android app can predict remaining battery life as accurately as the `PowerManager.getBatteryDischargePrediction()` system API introduced in Android 12. Smartphone battery data are intrinsically right-censored — users essentially never discharge to 0 % in normal use — so, following Li *et al.*, we adopt the Concordance index as primary metric. We passively collect 66,001 measurements over 45 days across four devices (Xiaomi and three Pixel models) and compare six methods: a quantised Conv1D, a Random Forest, a mean-predictor floor, a linear drain-rate baseline, an exponential fit, and the Google API. A *segment-level* train/test split prevents the sequence-shuffle leakage common in sliding-window setups; on our data the inflation it causes ranges from $\sim$1.24$\times$ (multi-device) to $\sim$1.69$\times$ (single-device subset), which we report as a methodological observation in its own right. On the leakage-free split, TinyML and Random Forest reach C=0.66 and C=0.68 against the training target y^{extrap}, both significantly above the floor ($p_{\text{BH}} < 0.001$ over 15 pairwise tests with Benjamini–Hochberg correction); against the uncensored measured target y^{real} neither ML model significantly outranks the constant mean. The analytical baselines and the Google API form a tied top group at $C \approx 0.77$, with Google not significantly better than linear extrapolation (ΔC=+0.009, p_{BH}=0.11). A per-device breakdown reveals strong hardware dependence: on the Pixel 7 Pro TinyML reaches C=0.75, on the Xiaomi only C=0.59. The TFLite INT8 pipeline itself meets its targets (14.4 kB, single-microsecond inference latency on a CPU reference). The bottleneck is sensor signal, not model capacity.**

***Index Terms*—TinyML, smartphone, battery prediction, on-device inference, Concordance index, data leakage, Android.**

I. Introduction

Predicting the remaining battery life of a smartphone is a long-standing problem [1], [2] with two qualitatively different solution classes. *Analytical* methods (e.g. the linear extrapolation used by most legacy "Settings → Battery" screens, or the runtime power-profiling of Kim *et al.* [1]) project the recently observed drain rate into the future. *Machine-learning* methods, in contrast, attempt to exploit context — which apps are active, how bright is the screen, what is the battery temperature — [2], [3].

Since Android 12 (October 2021), the platform exposes a system estimator via `PowerManager.getBatteryDischargePrediction()` [4], which returns a duration estimate computed inside the operating system. As a system process the estimator runs at a privilege level not available to third-party apps and can in principle access hardware counters that the Android permission model does not expose to user-installed applications. The companion method `isBatteryDischargePredictionPersonalized()` indicates whether the returned estimate has been adapted to the specific device's usage history.

Despite shipping since 2021, the accuracy of this system estimator has, to our knowledge, never been independently benchmarked against app-level alternatives on real-world data; nor does the TinyML literature (Section II) treat the smartphone battery-prediction case, focusing instead on microcontroller-class hardware. This twofold gap — an unquantified system baseline and an under-studied deployment target — is what the present paper addresses.

We ask: *how well can a TinyML model running inside an Android app predict remaining battery life, compared to analytical baselines and Google's native API — measured on accuracy and on-device efficiency?* TinyML here refers to small quantised neural networks (typically tens of kilobytes) designed for low-power edge devices [5]. While the TinyML literature largely targets microcontrollers, smartphones share two relevant properties: deployment constraints (model size matters for APK weight, latency matters for battery cost) and ubiquity.

This paper provides an empirical answer for the smartphone case. We report a six-way head-to-head comparison on identical real-world data (66,001 measurements collected over a 45-day window from four Android devices spanning two manufacturers), explicitly controlled for a class of data-leakage pitfalls that we found to inflate apparent results by a data-diversity-dependent factor (Section IV-A).

Contributions

1) **Methodological observation:** On sliding-window time-series data, a random shuffle of *sequences* (the default `train_test_split`) silently leaks future information into training. On our data the inflation factor depends on data diversity ($\sim$1.24$\times$ on the multi-device dataset, $\sim$1.69$\times$ on the single-device subset; see Table I). We propose and measure a segment-level split as the principled alternative.

2) **Multi-device six-way comparison with statistical rigour:** We collect data on four Android devices (Xiaomi 2107113SG, Pixel 7 Pro, 8 Pro and 9 Pro XL) and compare TinyML against four explicit baselines (Random Forest, mean predictor, linear drain rate, exponential fit) and the native Google API. Differences are reported with bootstrap 95% confidence intervals; pairwise significance uses a paired permutation test for MAE and a paired bootstrap on ΔC for the C-index (per-sample swap is invalid for a pairwise metric), with Benjamini–Hochberg correction over 15 tests per family.
3) **Device-dependent learnability:** Per-device evaluation reveals a strong hardware effect. On the Pixel 7 Pro the TinyML model achieves $C{=}0.75$, more than two and a half times the gap to the mean predictor that the same model attains on the Xiaomi ($C{=}0.59$). This demonstrates that the signal available to an app-level model is dominated by the underlying sensor stack, not by the model architecture or training budget.
4) **Model-independent feature diagnostics:** Beyond permutation-importance on the Random Forest, we report Pearson, Spearman and mutual-information statistics for all ten features. This separates a model-specific failure ("Conv1D is the wrong architecture") from a data-level limitation ("the publicly available features only carry enough signal on certain hardware").
5) **Reproducible open pipeline.** All code, configuration, raw data, intermediate artefacts, and the auto-generated report are released in a structured repository. A single command (`python run_pipeline.py`) reproduces every number in this paper.

II. Related Work

a) Smartphone battery prediction.: Li *et al.* [2] present, to our knowledge, the largest-scale empirical study on this problem to date: 51 users tracked over 21 months. They explicitly identify and address what they call a "severe data missing problem" — users very rarely discharge their phones to 0%, so the ground-truth remaining time at most observations is unobserved. Their evaluation is therefore based on the *concordance index* [6], which remains informative when the target is partially unobservable. We interpret this as a right-censoring setting and follow their methodological choice. Flores-Martin *et al.* [3] take a closely related path, comparing a feed-forward DNN against an LSTM on user-specific application, sensor and screen-on-time features collected from smartphones; the LSTM treats the data as a time series. Our contribution differs in three ways: (i) we add a TinyML-quantised on-device deployment, (ii) we evaluate against the native Google API and against analytical baselines on identical data, and (iii) we explicitly control for the segment-level leakage problem.

b) TinyML.: Banbury *et al.* [5] introduced MLPerf Tiny, which they describe as "the first industry-standard benchmark suite for ultra-low-power tiny [machine learning] systems" that measures *accuracy, latency, and energy*. We follow the first two and report energy-relevant proxies (model size and inference latency on a CPU reference); on-device energy measurement on commodity Android hardware is not exposed to third-party apps. Recent TinyML surveys [7], [8] focus primarily on microcontroller-class hardware (MCU, Arduino, Raspberry Pi); smartphone applications are not a central topic in either. We treat the smartphone case as under-studied within the TinyML literature on this empirical basis.

c) Time-series evaluation methodology.: Albelali and Ahmed [9] systematically study how the order of two preprocessing steps — forming sliding-window sequences first, then partitioning into train and test — affects LSTM forecasting evaluation. When sequences are constructed before the split, samples derived from the same underlying segment can end up on opposite sides of the partition, causing systematic optimism. They report an "RMSE Gain" (relative increase due to leakage) of up to 20.5 % specifically for 10-fold cross-validation at extended lag steps, and below 5 % for 2-way and 3-way holdout splits. We replicate the direction of their observation in our sliding-window setting, although the magnitude we observe is substantially larger than theirs (Section IV-A); we report this as a domain-specific empirical anchor rather than as a replication of their precise quantitative result.

III. Methodology

A. Data Collection

A foreground service samples one feature vector every 30 s on each device. Data are collected on four Android devices spanning two manufacturers:

- **Xiaomi 2107113SG** (Android 12+, 8 GB RAM): 38,087 samples, 69 sessions.
- **Google Pixel 7 Pro**: 16,919 samples, 72 sessions.
- **Google Pixel 8 Pro**: 3,354 samples, 18 sessions.
- **Google Pixel 9 Pro XL**: 7,641 samples, 21 sessions.

Total: 66,001 measurements, 180 sessions, a 45-day total collection window (25 March–10 May 2026) with per-device active windows of 21–34 days. The single-vendor (Pixel) majority is intentional: Pixel devices are the reference platform for Google's `getBatteryDischargePrediction()` API, so this group provides the most favourable conditions for the system baseline.

Each feature vector contains ten features that are accessible without any privileged permission beyond `PACKAGE_USAGE_STATS`: battery level, screen on/off, brightness, currently active app category, Wi-Fi state, mobile-data state, charging state, CPU usage (estimated from average core frequency, since `/proc/stat` is restricted in modern Android), battery temperature and Wi-Fi-hotspot state. Two reference values are logged alongside but *not* used as features: `system_estimate_min`, the value returned by Google's `getBatteryDischargePrediction()`, and `system_personalized`, the boolean indicator returned by `isBatteryDischargePredictionPersonalized()`. The collector also logs the per-step prediction of our own TinyML model (when available) and a naive linear

baseline computed from `BatteryManager` counters (`CHARGE_COUNTER`/`CURRENT_AVERAGE`).

B. Discharge Segments and Targets

We partition the raw CSV into *discharge segments*: maximal contiguous runs of `charging=0` within the same session, split again whenever consecutive samples are more than five minutes apart, and filtered to a minimum length of $L_{\min}$=15 samples. From the 66,001 measurements across the four devices (180 sessions) the procedure yields 553 valid discharge segments.

For each sample i in a segment with last-sample timestamp t_N, last battery level b_N, total drain $\Delta b = b_0 - b_N$ and total duration Δt, we compute two regression targets:

$$y_i^{\text{real}} = (t_N - t_i)/3.6 \times 10^6 \text{ms/h}, \quad (1)$$

$$y_i^{\text{extrap}} = y_i^{\text{real}} + b_N \cdot \Delta t/\Delta b. \quad (2)$$

The first target is the directly observed remaining time within the segment. It is structurally censored (we typically stop at $b_N > 0$, so the true remaining time is unobserved). The second target adds an extrapolation to $b = 0$ assuming the average drain rate of the segment, and matches in spirit what every method in our comparison conceptually predicts. We use y^{extrap} as the common evaluation target in Section IV; y^{real} is reported for context.

C. Sliding Windows and Segment-Level Split

We slide a window of length W=10 over each segment, producing 20,842 sequences in total. The training target attached to each window is the value at the most recent timestep within the window.

Crucially, the train/val/test split is performed over *segments*, not over *sequences*. Every sliding window from a given segment is assigned, as a unit, to exactly one of train/val/test according to a fixed seed. Concretely: 387 segments (13,901 sequences) to train, 83 (3,056) to val, and 83 (3,885) to test. A naive shuffle of the 20,842 sequences would distribute samples from the same segment across the splits and leak the segment's local trajectory — the kind of failure mode characterised by Albelali and Ahmed [9]. We measured the magnitude of this leakage by training the same Conv1D model under both splitting strategies; the results are reported in Section IV-A.

D. Methods Compared

All six methods predict the remaining time in hours.

- **TinyML Conv1D.** Two 1D convolutional layers (32 filters, kernel 3, same-padding, ReLU) with global average pooling, followed by a dense layer (32 units, ReLU), dropout (p=0.2), a second dense layer (16 units, ReLU), and a linear head. Total: 5,697 parameters. Trained with Adam (η=5×10^{-4}), MSE loss, batch size 32 and early-stopping on validation loss (patience 20). StandardScaler is fit on the training sequences and applied to all splits. After training, the Keras model is converted to three TFLite variants (dynamic-range, float16, full-int8). The full-int8 variant is the deployed artefact in the companion app.
- **Random Forest sanity model.** A scikit-learn `RandomForestRegressor` [10] (200 trees) trained on the flattened sliding window (a 10×10 tensor reshaped into a 100-dimensional vector). The Random Forest is a fundamentally different modelling paradigm and is included to disentangle model-specific failure from data-level failure: if the Random Forest also fails to outperform the mean predictor, the bottleneck is the data, not the choice of Conv1D.
- **Mean predictor (floor).** For every test point, predicts the training-set mean of y^{extrap} (11.18 h). By construction, its Concordance index is exactly 0.5 and any learning model that does not significantly exceed this floor has not learned a feature-to-output relationship.
- **Linear drain-rate baseline.** For each test sample, computes the drain rate $\hat{r}$=$\Delta b/\Delta t$ over the last 10 raw samples of the same discharge block, and predicts $\hat{y}$=$b_t/\hat{r}$. This is the same calculation that legacy "Settings → Battery" screens implement on top of the `BatteryManager` counters.
- **Exponential fit.** Fits $b(\tau)$=a+$c\,e^{-k\tau}$ to the most recent 10 raw samples and extrapolates to b=0, falling back to the linear baseline if the fit diverges or the extrapolated zero-crossing lies in the past. This adds a single nonlinear term beyond the linear baseline.
- **Google API.** Reads `system_estimate_min` from the CSV and converts to hours. The value is undefined while charging or before the system has accumulated enough history; samples without a defined value are excluded from coverage-aware tables (Section IV).

E. Metrics and Statistical Tests

For each method we report mean absolute error (MAE), root mean square error (RMSE), mean error (ME, a bias indicator), Concordance index [6] (C), and accuracy thresholds ($\text{Acc}_{\pm1\text{h}}$, $\text{Acc}_{\pm2\text{h}}$). Following Li *et al.* [2], we treat the Concordance index as the primary metric, since it is invariant to target-bias and remains meaningful under censoring.

We attach 95% bootstrap confidence intervals to MAE, RMSE, ME and C (1,000 resamples; 200 for C, which is $O(n^2)$). Pairwise method differences use two different paired tests, chosen to match each metric's structure. *For MAE*, which is sample-additive, we use a paired permutation test (1,000 permutations): per-sample, with probability 0.5, swap the two methods' predictions, recompute the metric difference, and report the share of permutations whose absolute difference equals or exceeds the observed one ($(k$+$1)/(B$+$1)$ correction). *For C-index*, which is a pairwise metric, the per-sample swap is not valid because it destroys the pair structure the metric depends on; we instead use a paired bootstrap on ΔC (500 resamples on the joint $(y^{\text{true}}, \hat{y}_A, \hat{y}_B)$ triples), giving a 95% CI for ΔC and a two-sided bootstrap p-value $2\min(\Pr[\Delta C \le 0], \Pr[\Delta C \ge 0])$. With six methods, 15 pairwise tests are run per metric per target. We report both

TABLE I

EFFECT OF TRAIN/TEST SPLIT STRATEGY ON APPARENT TINYML PERFORMANCE. SAME MODEL, SAME HYPERPARAMETERS IN BOTH ROWS OF EACH BLOCK; THE ONLY CHANGE IS WHETHER SLIDING-WINDOW SEQUENCES ARE SHUFFLED FREELY (LEAKY) OR WHETHER THE SPLIT IS PERFORMED AT THE SEGMENT LEVEL (CLEAN). THE INFLATION FACTOR IS THE RATIO OF CLEAN TO LEAKY TEST MAE.

Dataset	Seqs	MAE leaky	MAE clean	Inflation
Single device (Xiaomi only)	9,024	6.53	**11.06**	$\sim$1.69$\times$
Multi-device (4 devices)	20,842	4.00	**4.97**	$\sim$1.24$\times$

raw p-values and Benjamini–Hochberg FDR-adjusted p-values (p_{BH}), and base significance claims on the adjusted values.

IV. RESULTS

A. Effect of the Splitting Strategy

The same Conv1D model, trained on the same data with the *only* change being how the sliding-window sequences are split into train/val/test, yields the result in Table I. We report both the single-device subset (Xiaomi only, used during initial prototyping) and the full multi-device dataset.

Two points are worth flagging. First, both rows show that the leakage-free segment-level split produces a higher (i.e. honest) test MAE, consistent in direction with the LSTM-benchmark observation of Albelali and Ahmed [9]. Second, the magnitude of the inflation is itself dataset-dependent: the single-device subset shows a noticeably larger inflation ($\sim$1.69$\times$, or a $\sim$69 % relative MAE increase) than the multi-device dataset ($\sim$1.24$\times$, $\sim$24 %). The multi-device number is in the same broad range as the $\sim$20.5 % RMSE Gain that Albelali and Ahmed [9] report for 10-fold cross-validation at extended lag steps; the single-device number is larger by a factor of $\sim$3.4. We read this as evidence that data diversity attenuates the leakage effect, presumably because intra-segment correlations become a smaller share of the total signal. The practical recommendation stands either way: *report the splitting strategy explicitly and verify it is segment-level*. All subsequent results in this paper use the leakage-free multi-device split.

B. Six-Way Accuracy on the Common Subset

Table II reports MAE and C (with 95% bootstrap CIs) on the common subset of 2,883 test sequences (out of 3,885) for which all six methods produce a defined prediction. Against the training target y^{extrap}, the two ML methods clear the mean-predictor floor: TinyML reaches C=0.656 [0.647, 0.664], Random Forest C=0.685 [0.673, 0.695], both with confidence intervals that exclude the floor of 0.500. The two analytical baselines and the Google API form a tied top group at $C \approx 0.77$, and the Google API is *not significantly better than the linear drain-rate baseline* on C or MAE (Table IV).

Caveat on the common subset. Restricting the comparison to samples where every method (notably the Google API) is defined introduces a mild selection effect: the mean of y^{extrap} on the common subset is 6.76 h versus 7.68 h on the full test set. The Google API is more often available at shorter remaining times, so the common subset is biased toward easier prediction targets. This affects all methods symmetrically (we compare them on the same samples), so the ranking is unaffected, but absolute MAE values would be larger on the unrestricted test set.

C. Per-Device Behaviour

The aggregate ranking in Table II hides a substantial between-device variation, summarised in Table III. On the Pixel 7 Pro, the TinyML model reaches C=0.75, comparable to the analytical baselines and 0.10 below the Google API's C=0.85 on that device. On the Xiaomi 2107113SG, the same model collapses to C=0.59. Random Forest behaves similarly: 0.81 on Pixel 7 Pro, 0.63 on Xiaomi. The two analytical baselines, by contrast, are stable in the 0.65–0.79 band across all four devices.

Two observations on Table III are worth flagging separately. First, on the Pixel 9 Pro XL the simple linear drain-rate baseline (C=0.730) outperforms the Google API (C=0.677), inverting the usual expectation. Second, the Pixel 8 Pro shows the largest divergence between C and MAE: the Google API ranks samples almost perfectly (C=0.92) but is heavily biased on absolute remaining time (MAE = 9.92 h). n=98 for that device makes both numbers correspondingly noisy. We treat this as evidence against any monotonic "ML always wins" narrative on this problem and as a justification for reporting both ranking and absolute-error metrics throughout.

D. Statistical Significance

The pairwise tests (Table IV) reinforce the visual picture from Table II. Against the training target y^{extrap}, both ML models clear the mean-predictor floor on C (BH-adjusted $p_{\text{BH}} < 0.001$). Within the top group, Linear and Google are statistically indistinguishable (ΔC=+0.009, p_{BH}=0.11), as are Linear and Exponential (ΔC=+0.004, p_{BH}=0.27) and Exponential and Google (ΔC=+0.005, p_{BH}=0.28). The Google API significantly exceeds both ML models on C ($p_{\text{BH}} < 0.001$).

a) Robustness check against the measured target.: Re-running the same tests against y^{real} (uncensored measured remaining time) materially changes two of the comparisons in a way that *strengthens* the negative-result reading: TinyML *vs.* Mean collapses to ΔC=+0.001 (p_{BH}=0.82, ns), and Random Forest *vs.* Mean reverses to ΔC=−0.052 ($p_{\text{BH}} < 0.001$). In other words, on the measured target the two ML models do *not* outperform a constant-mean baseline at ranking (RF in fact ranks worse), while the Google API and the analytical baselines retain their ranking advantage. We treat y^{extrap} as the primary target for significance because it is the training target and matches the bias of the ML models; the y^{real} result is the more honest reading of practical utility.

E. Cumulative Error Distribution

Figure 1 shows the share of test predictions whose absolute error against y^{real} does not exceed a given tolerance, for each method that produces enough valid samples.

TABLE II

ACCURACY ON THE COMMON SUBSET (N=2,883 TEST SEQUENCES, ALL SIX METHODS DEFINED). TARGET: y^{EXTRAP} AS DEFINED IN EQ. (2). BRACKETS ARE 95% BOOTSTRAP CIS.

Method	MAE (h)	RMSE (h)	ME (h)	C-index	$\text{Acc}_{\pm 2\text{h}}$
Mean predictor (floor)	6.52 [6.29, 6.76]	9.25 [8.69, 9.80]	+4.42	0.500 [0.500, 0.500]	21.5 %
TinyML Conv1D	4.60 [4.33, 4.82]	8.47 [7.78, 9.13]	+1.81	0.656 [0.647, 0.664]	43.7 %
Random Forest	4.06 [3.83, 4.28]	7.56 [6.98, 8.11]	+1.61	0.685 [0.673, 0.695]	46.4 %
Linear (drain rate)	**3.30 [3.03, 3.57]**	8.57 [7.69, 9.39]	−2.39	**0.770 [0.761, 0.780]**	57.3 %
Exponential fit	3.63 [3.34, 3.91]	9.04 [8.14, 9.84]	−1.60	0.767 [0.758, 0.776]	59.0 %
Google API	3.37 [3.10, 3.65]	8.65 [7.74, 9.45]	−2.37	0.762 [0.751, 0.772]	59.0 %

TABLE III

PER-DEVICE PERFORMANCE AGAINST y^{EXTRAP}. n IS THE DEVICE-SPECIFIC TEST-SET SIZE. PAIRS OF COLUMNS SHOW C-INDEX (TOP METRIC) AND MAE IN HOURS (BOTTOM METRIC). THE MAE COLUMN REVEALS THAT, EVEN WHEN C IS HIGH, ABSOLUTE CALIBRATION CAN BE POOR (E.G. THE GOOGLE API ON PIXEL 8 PRO: C=0.92 BUT MAE = 9.92 h). THE TWO ML MODELS SHOW THE LARGEST SWING ACROSS DEVICES.

Device	n	TinyML C	TinyML MAE	Random Forest C	Random Forest MAE	Linear C	Linear MAE	Google API C	Google API MAE
Pixel 7 Pro	1,825	0.754	2.50	0.807	2.04	0.792	2.05	**0.853**	**1.89**
Pixel 8 Pro	98	0.742	2.67	0.815	2.94	0.696	**1.36**	**0.921**	9.92
Pixel 9 Pro XL	614	0.571	2.37	**0.767**	**1.62**	0.730	2.54	0.677	2.53
Xiaomi 2107113SG	1,348	0.593	10.17	0.628	9.94	**0.724**	**5.87**	0.473	8.54

TABLE IV

SELECTED PAIRWISE TESTS, COMMON SUBSET (N=2,883), TARGET y^{EXTRAP}. MAE USES A PAIRED PERMUTATION TEST (1,000 PERMUTATIONS); C-INDEX USES A PAIRED BOOTSTRAP ON ΔC (500 RESAMPLES) BECAUSE THE PER-SAMPLE SWAP IS INVALID FOR A PAIRWISE METRIC. p_{BH} IS THE BENJAMINI–HOCHBERG-ADJUSTED p-VALUE OVER THE 15 PAIRWISE TESTS IN EACH FAMILY. "NS" = NOT SIGNIFICANT AT p_{BH}>0.05.

Comparison	C_A	C_B	ΔC	p_{BH}
TinyML *vs.* Mean	0.656	0.500	+0.156	<**0.001**
RF *vs.* Mean	0.685	0.500	+0.184	<**0.001**
TinyML *vs.* RF	0.656	0.685	−0.028	<**0.001**
Linear *vs.* Exponential	0.770	0.767	+0.004	0.27 (ns)
Linear *vs.* Google	0.770	0.762	+0.009	0.11 (ns)
Exp. *vs.* Google	0.767	0.762	+0.005	0.28 (ns)
TinyML *vs.* Google	0.656	0.762	−0.105	<**0.001**
RF *vs.* Google	0.685	0.762	−0.077	<**0.001**

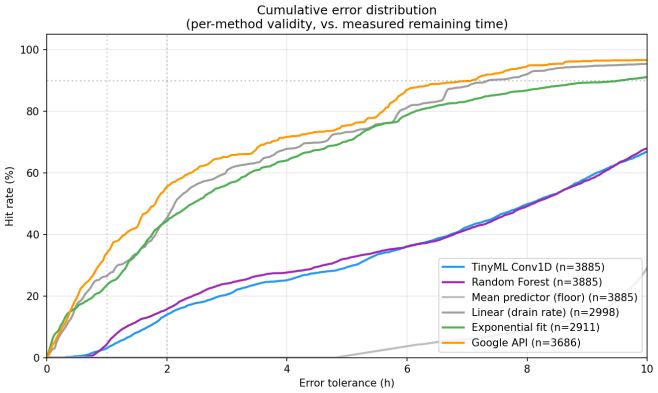

Fig. 1. Cumulative error distribution against the (uncensored) measured remaining time y^{real}. The analytical baselines (linear, exponential) are particularly competitive at small tolerances because the test set is dominated by short measured remaining times (see Fig. 2 and Section VI).

F. Model-Independent Data Diagnostics

Table V reports Pearson, Spearman and mutual-information statistics between each feature (taken at the latest timestep of the window) and y^{extrap}, computed on the training split (n=13,901). Linear (Pearson) correlations are moderate, with `hotspot_on`, `wifi_on` and `mobile_data_on` all showing $|r|\approx 0.5$; these three features encode connectivity modes that correlate strongly with device-specific usage patterns rather than with discharge dynamics directly. Mutual information identifies `temperature` (2.07), `battery_level` (1.72) and `brightness` (1.14) as the features carrying the most nonlinear signal; all other features fall below 0.5. Only one feature is effectively constant in our discharge-only filter: `charging`, which is 0 by construction.

The non-trivial linear correlations on the three connectivity features reflect the multi-device structure of our training set: Xiaomi sessions have systematically different WiFi/mobile-data patterns than the three Pixel devices, and battery temperature distributions also differ. These features are therefore informative as device identifiers rather than as direct predictors of discharge dynamics, which limits how well a model can generalise across hardware. We return to this point in Section VI.

a) Permutation-importance check (Random Forest).: The same conclusion holds when measured through model-side feature importance. Permutation importance on the held-out test set (device-stratified subsample, n_{repeats}=5, scoring on −MAE) ranks `brightness` (ΔMAE=+0.05 h) and `hotspot_on` (+0.04 h) as the only features whose per-

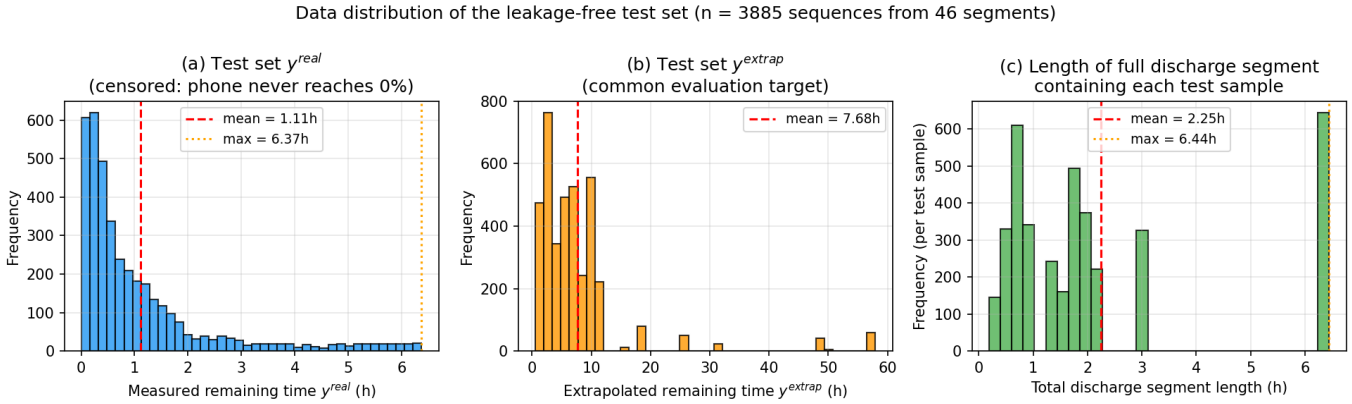

Fig. 2. Distribution of (a) measured remaining time y^{real}, (b) extrapolated target y^{extrap}, and (c) total discharge segment length on the leakage-free multi-device test split. Panel (a) makes the right-censoring concrete: the test set y^{real} has mean 1.11 h and maximum 6.37 h, since users do not discharge their phones to 0 % in normal use. Panel (b) shows that the extrapolated target spreads across ∼0–58 h, which is the regime in which all six methods actually attempt to predict.

TABLE V
FEATURE–TARGET RELATIONSHIPS ON THE TRAINING SPLIT (N=13,901), TARGET: y^{EXTRAP}. SORTED BY $|r|$ DESCENDING.

Feature	Pearson r	Spearman ρ	MI (nat)
`hotspot_on`	+0.53	+0.36	0.23
`wifi_on`	+0.51	+0.39	0.24
`mobile_data_on`	−0.50	−0.39	0.24
`battery_level`	+0.43	+0.58	**1.72**
`screen_on`	−0.35	−0.25	0.16
`temperature`	−0.24	−0.26	**2.07**
`active_app_category`	−0.10	−0.06	0.29
`cpu_usage`	−0.05	+0.02	0.31
`brightness`	−0.02	+0.08	**1.14**
`charging` (constant)	0.00	0.00	0.00

mutation *worsens* the model's predictions; all other features have small or even negative aggregated importance. The negative aggregated values for `battery_level` (−5.3 h), `temperature` (−0.4 h) and `cpu_usage` (−0.2 h) indicate that the Random Forest is fitting these features to noise on the training set in a way that does not transfer to test (consistent with the train/val MAE ratio of ∼4.6 in our regularised RF, despite `max_depth`=20 and `min_samples_leaf`=5). The fact that even the most useful single feature changes the test MAE by less than 0.1 h — on a target with mean 6.76 h on the common subset — is itself the central observation: no single feature available to an app-level model carries enough signal to push the model's MAE meaningfully closer to the analytical baselines.

G. Per-Bucket Behaviour

A bucket-level analysis on battery-level at prediction time (common subset, target y^{extrap}) shows that the Google API's advantage is not uniform but concentrated in the longer horizons. At battery 75–100 %, where the longest remaining-time predictions sit, Google reaches C=0.71 and Linear C=0.71 versus 0.52 (TinyML) and 0.57 (RF). At battery 50–75 %, Google and the analytical baselines compress to C≈0.62–0.63 and the gap to the ML models is smaller. At battery 0–25 % (near-empty, remaining time short), Random Forest unexpectedly leads (C=0.74) while Linear, Exponential and Google cluster in the 0.52–0.57 band and TinyML drops below the mean-predictor floor (C=0.34). Bucketed on segment length, the 2–5 h bucket (n=820 common) is where Google attains its peak C=0.96 on y^{extrap}, with all other methods at C=0.85–0.90.

TABLE VI
MODEL SIZE AND INFERENCE LATENCY ON THE DEVELOPMENT MACHINE (CPU). LATENCY IS THE AVERAGE OVER 1,000 SINGLE-SAMPLE RUNS AFTER 50 WARM-UP RUNS.

Variant	Size (KB)	Avg latency
Keras Float32	109.19	39.0 ms
TFLite dynamic-range	15.99	3.1 µs
TFLite float16	17.80	2.9 µs
TFLite full-INT8 (deployed)	**14.35**	3.3 µs

H. Efficiency

Table VI reports model size and inference latency for the four model artefacts, measured on the development machine (CPU only, 1,000 inference repetitions per variant after 50 warm-up runs). The full-INT8 TFLite variant compresses the Keras model from 109 kB to 14.4 kB (7.6×) and reduces single-inference latency from 39.0 ms to 3.3 µs (∼12,000×). The TinyML *technique* works as advertised; only the predictive value of the quantised model on this dataset does not.

V. DISCUSSION

Four findings deserve emphasis.

a) The leakage observation generalises but is data-diversity-dependent.: Table I shows that the leakage effect is consistent in direction (random shuffle always produces an over-optimistic test MAE) but variable in magnitude. On the

multi-device dataset the inflation is $\sim$1.24$\times$, in the same broad range as the $\sim$20.5 % RMSE Gain Albelali and Ahmed [9] report for 10-fold cross-validation at extended lag steps. On the much smaller single-device subset the inflation is $\sim$1.69$\times$. A plausible reading is that as the dataset accumulates more segments and more device-to-device variation, intra-segment correlations make up a smaller fraction of the available signal and the leak provides correspondingly less spurious help. The practical takeaway is the same in either regime: any sliding-window time-series experiment in this domain that does not specify the splitting strategy is suspect. We recommend that future work in app-based mobile sensing report the splitting unit (sequence, segment, session, or user) explicitly.

b) Both ML models learn target-specific signal, less so on the measured target.: On the multi-device test set against the training target y^{extrap}, TinyML reaches C=0.66 and Random Forest C=0.68, both above the mean-predictor floor at $p_{\text{BH}} < 0.001$. Against the uncensored measured target y^{real}, the same models reduce to ΔC=+0.001 (p_{BH}=0.82, ns) and ΔC=−0.05 (p_{BH}<0.001) respectively, i.e. no significant ranking advantage over the constant mean (Random Forest in fact ranks worse). The analytical baselines (Linear drain rate at C=0.77, Exponential fit at C=0.77) outperform both ML models substantially. The most parsimonious explanation is that the dominant signal in app-level battery data is the recent drain rate, which the analytical baselines exploit directly while the ML models have to recover it from ten weakly correlated features. With a richer feature set — in particular instantaneous current — the gap might close, but on publicly accessible sensors there is little visible benefit from a learned model over $b/\dot{b}$ extrapolation.

c) Google's API is on par with linear extrapolation on the typical horizon.: On the common subset, the Google API is statistically *indistinguishable* from the linear drain-rate baseline both on MAE (Δ=+0.07 h, p_{BH}=0.16) and on C (ΔC=+0.009, p_{BH}=0.11). This aggregate result, however, hides a structure that only emerges in a bucket analysis: of the 2,883 common-subset samples, 1,376 (47.7 %) fall in the y^{extrap} <5 h bucket where Linear and Google produce nearly identical errors (MAE 1.11 h vs. 1.02 h), and 1,414 (49.0 %) fall in the 5–15 h bucket where they remain close. Only 93 samples (3.2 %) fall above 15 h, and in the very-long-horizon bucket ($\geq$30 h, n=58) the Concordance index diverges sharply: Google reaches C=0.98 while the linear baseline collapses to C=0.64. The system API's privilege — access to per-application power accounting and the Adaptive Battery system, which a third-party app cannot reach — does translate into a measurable advantage, but only where the prediction horizon is long enough that a simple current-based extrapolation breaks down. On the short-to-medium horizons that dominate everyday usage, the linear baseline is competitive. This is good news for app developers: BatteryManager counters alone are enough for the use cases users actually see.

d) Predictability is hardware-dependent — with a confound.: The per-device breakdown (Table III) is the strongest practical finding. The same TinyML model reaches C=0.75 on Pixel 7 Pro and C=0.59 on Xiaomi — a ΔC of 0.16, roughly 46 % of the available range between the mean-predictor floor and the best per-device score. Random Forest shows the same pattern; the two analytical baselines are less device-sensitive (Linear $C \in [0.70, 0.79]$, Exponential $C \in [0.65, 0.79]$ across all devices). A confounder we cannot fully isolate is that the per-device collections differ in input-feature distributions: on Xiaomi the battery level is above 75 % in 88.8 % of measurements (mean 94 %), against 40.8 % on Pixel 7 Pro (mean 58 %). The Xiaomi sub-collection therefore offers fewer discharge dynamics for an ML model to learn from, and the Xiaomi test set has a longer mean y^{extrap} (10.14 h vs. 5.79 h on Pixel 7 Pro). Part of the ΔC=0.16 swing therefore likely reflects this distribution difference rather than sensor quality alone; disentangling the two would require a controlled identical-usage protocol across devices, or much larger per-device samples. The practical consequence is that any deployed app-level battery predictor should report a device-specific confidence estimate — a globally-trained model has very different reliability profiles on Pixel and Xiaomi hardware.

A. Where App-Level TinyML for Battery Prediction Still Makes Sense

Our negative finding applies specifically to *modern Android devices ($\geq$ API 31) where the system already provides a privileged ML prediction*. It does not generalise to four scenarios where app-level TinyML remains a defensible engineering choice: (i) **pre-Android-12 devices**, where the only system baseline is legacy linear extrapolation rather than a learned predictor, so an app-level model competes only against `Linear (drain rate)`; (ii) **custom ROMs and de-Googled Android** (LineageOS, GrapheneOS, /e/OS), which ship without the proprietary services that implement `getBatteryDischargePrediction()`, leaving even API-31+ devices on the legacy estimator; (iii) **wearables** (smartwatches, fitness bands) running on constrained hardware ($\leq$1 GB RAM, Cortex-M-class co-processors, sub-500 mA h batteries), the form factor TinyML was originally designed for, where the 14 KB INT8 model would fit even on bare microcontrollers; and (iv) **IoT and embedded battery-powered devices** (sensor nodes, environmental monitors, asset trackers [5], [8]) which both lack a privileged system predictor and have fundamentally different physical constraints (continuous monotonic discharge under known load profiles), so the data-signal limitations documented here for the unpredictable consumer-smartphone setting do not necessarily transfer.

We therefore read our results not as an argument against TinyML in general, but as evidence that on modern Android with the Google API present, the sandbox-vs-system access asymmetry dominates whatever benefit a small on-device model could add. The methodology developed here (segment-level splits, mean-predictor floor, model-independent feature diagnostics, paired bootstrap on ΔC with BH correction) transfers to all four scenarios above, where it can settle each case empirically rather than by analogy.

VI. Limitations and Scope

A. Right-censored data is intrinsic to the problem

Our test set is censored: as Fig. 2(a) shows, the mean measured remaining time y^{real} is 1.11 h and the maximum is 6.37 h. Crucially, this is not a peculiarity of our collection window but a structural property of the smartphone battery-prediction problem. Smartphones in normal use are essentially never discharged to 0 %. Li *et al.* [2], working at a vastly larger scale of 51 users over 21 months, characterise this as a "severe data missing problem" and adopt the Concordance index as their evaluation metric, since it remains informative when the target variable is not directly observed at many measurement points.

We adopt the same convention. A study that wished to remove the censoring would need a controlled discharge protocol (a phone forced from ∼80 % to 0 % under a scripted load) which by construction generalises only to that protocol, not to organic usage. The censoring in our test set is therefore best read as the natural boundary of any passive observational study in this domain, rather than as a defect of our particular collection.

B. Limited multi-device sample and no cross-device generalisation test

Data come from four devices spanning two manufacturers (Xiaomi and Google). Within the Pixel family, only one device per generation (7 Pro, 8 Pro, 9 Pro XL) is sampled. The strong device-dependence we observe (Section IV-C) means that the absolute C values for any single device should be read with appropriate caution. We expect the qualitative ordering — analytical baselines ≈ Google API > ML models > mean predictor — to be robust, since it survives intact on every device with $n \geq 600$ test samples (Pixel 7 Pro, Pixel 9 Pro XL, Xiaomi). The thinnest cell in our table is Pixel 8 Pro with only n=98 test samples; the C estimates there have correspondingly wide confidence intervals.

Importantly, our train/val/test split distributes *segments* randomly across the four devices, so the trained models have seen samples from every device during training. The per-device numbers in Table III therefore measure *within-device* generalisation only. We do not test how the trained models would perform on a fifth, unseen device; a leave-one-device-out evaluation is the natural follow-up.

One per-device anomaly is worth flagging explicitly: on the Pixel 9 Pro XL the TinyML model collapses to C=0.57 while the Random Forest on the same samples reaches C=0.77 and the linear drain-rate baseline C=0.73. We do not have an instrumentation-level explanation. Plausible contributing factors are that (i) the Pixel 9 Pro XL sensor stack differs from the older Pixel generations in ways our ten-feature representation does not capture, and (ii) the global StandardScaler may suit Conv1D worse than RF on this specific feature distribution. We flag this as an open question rather than paper over it.

C. One feature lacks within-discharge variance

After filtering to discharge-only samples, `charging` is by construction 0 everywhere in our training data and therefore carries no signal; we report it as constant in Table V for transparency. The other connectivity features (`wifi_on`, `mobile_data_on`, `hotspot_on`) do show variance across the multi-device dataset, but as Table V makes clear, their correlation with y^{extrap} is largely mediated by device identity rather than by causal coupling to discharge dynamics. A single-device subset would collapse most of this signal as well.

D. Several features are device-concentrated or rare

Beyond the constant `charging` feature, several other features in our dataset are not well-balanced. Across the 39,104 discharge data points in the merged dataset:

- `hotspot_on` is positive in 4.6 % of samples (1,816 of 39,104); 54 % of those positive samples come from the Pixel 8 Pro (978 of 1,816 across all devices).
- `wifi_on` is positive in 4.2 % of samples (1,643 of 39,104); the Pixel 8 Pro contributes the overwhelming majority of WiFi-enabled samples (1,475 of 1,643, i.e. 89.8 % of all positives), while the other three devices stay below 1 % WiFi each.
- `active_app_category` is heavily skewed: the *gaming* category (11.4 % of samples) is essentially Xiaomi-only (4,433 of 4,442 gaming samples). The *browser* (3.0 %) and *productivity* (0.9 %) categories together account for fewer than 4 % of discharge data and are also unevenly distributed across devices.

Practically, this means that the linear correlations on the connectivity features (Table V) and the permutation-importance ranking of `hotspot_on` (Section IV-F) reflect device-specific usage patterns rather than universal discharge-rate signals. A model that learns "hotspot on ⇒ Pixel 8 Pro-like discharge" is fitting device identity, not battery dynamics; it will not transfer to a fifth device that runs hotspot heavily but has a different sensor stack. We treat the observed feature signal as a soft upper bound on what an app-level model can learn from public sensors — and even that bound is reached only because of incidental between-device balance differences. A balanced multi-device deployment would shrink the apparent signal further.

E. What a definitive follow-up would look like

A study that aims to refute or confirm our central finding under more favourable conditions should combine three additions: a multi-device deployment ($N \geq 5$ phones across at least two manufacturers); a small number of controlled full-discharge cycles per device, providing uncensored ground truth for a confirmatory subset; and a longer observation window (∼3 months) capturing more behavioural variation. The collector app already logs the additional columns required for such a study (`system_personalized`, `own_prediction_h`, `linear_baseline_h`); only the deployment logistics remain.

VII. Conclusion

Our research question asked how well an app-level TinyML model can predict remaining battery life compared to analytical baselines and the native Android 12 prediction API, on the two axes of *accuracy* and *efficiency*. The four-device leakage-free evaluation gives a separable answer on each axis.

a) Accuracy.: TinyML and Random Forest reach $C{=}0.66$ and $C{=}0.68$ against the training target y^{extrap} ($p_{\text{BH}} < 0.001$ over the 15-test family), suggesting that publicly accessible smartphone sensors carry some exploitable signal. The strength of that conclusion is target-sensitive: against the uncensored measured target y^{real} neither ML model significantly outranks the constant mean ($\Delta C{=}{+}0.001$, $p_{\text{BH}}{=}0.82$ for TinyML; Random Forest's C actually falls below the mean baseline at $\Delta C{=}{-}0.05$, $p_{\text{BH}}{<}0.001$). The two ML models do not match the top group either way: the linear drain-rate baseline, the per-segment exponential fit, and the Google API are statistically tied at $C \approx 0.77$, and the Google API is indistinguishable from the simple linear baseline both on MAE ($p_{\text{BH}}{=}0.16$) and C ($\Delta C{=}{+}0.009$, $p_{\text{BH}}{=}0.11$) at the aggregate level. A bucket analysis (Section IV-C and Discussion) shows that this tie reflects the dominance of short and medium horizons ($\sim$97 % of test samples have $y^{\text{extrap}} < 15\,\text{h}$); at very long horizons ($\geq 30\,\text{h}$, $n{=}58$ on the common subset) the Google API substantially outperforms the linear baseline in C.

b) Efficiency.: The TFLite quantisation pipeline behaves exactly as advertised. The deployed full-INT8 model occupies $14.4\,\text{kB}$ and runs in single-microsecond latency on a CPU reference — a $7.6\times$ size reduction and a $\sim$12,000$\times$ latency reduction over the Keras Float32 reference. There is no meaningful efficiency cost to deploying the TinyML model; the open question on the accuracy axis is the only relevant trade-off.

c) Hardware sensitivity.: Per-device evaluation (Table III) shows a strong device dependence: the same TinyML model reaches $C{=}0.75$ on Pixel 7 Pro and $C{=}0.59$ on Xiaomi. Random Forest behaves identically, while the analytical baselines are far more device-stable. The most parsimonious reading is that the limiting factor is the available dynamic range rather than the model architecture — on Xiaomi the battery sits above 75 % in 88.8 % of measurements, leaving little observed discharge for any learning model to exploit. Any deployed app-level battery predictor should therefore report a device-specific confidence estimate.

The methodological observation — that the choice of splitting unit (sequence vs. segment) changes the apparent test MAE on this kind of data by a data-diversity-dependent factor of $\sim$1.24$\times$ to $\sim$1.69$\times$ — is, we believe, the more durable contribution. We release the complete pipeline, raw data and auto-generated reproducibility report so that future work can extend the comparison (more devices, prospective full-cycle data) or simply verify that random-shuffle benchmarks in this domain are not measuring what they appear to measure.

Reproducibility Statement

The pipeline (data preparation, training, TFLite conversion, evaluation and report generation) is available at the project repository. All hyperparameters are stored in `configs/default.yaml`. A full reproduction is invoked by `python run_pipeline.py` and typically completes in under twenty minutes on a laptop CPU (the evaluation stage with bootstrap intervals dominates the runtime). The auto-generated report contains every numerical value quoted in this paper.

References

[1] D. Kim, Y. Chon, W. Jung, Y. Kim, and H. Cha, "Accurate prediction of available battery time for mobile applications," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 15, no. 3, pp. 1–24, 2016.

[2] H. Li, X. Lu, X. Liu, T. Xie, K. Bian, F. X. Lin, Q. Mei, and F. Feng, "Predicting smartphone battery life based on comprehensive and real-time usage data," *arXiv preprint arXiv:1801.04069*, 2018. [Online]. Available: https://arxiv.org/abs/1801.04069

[3] D. Flores-Martin, S. Laso, and J. L. Herrera, "Enhancing smartphone battery life: A deep learning model based on user-specific application and network behavior," *Electronics*, vol. 13, no. 24, p. 4897, 2024.

[4] Android Open Source Project, "PowerManager.getBatteryDischargePrediction()," 2021, android API level 31 (Android 12). [Online]. Available: https://developer.android.com/reference/android/os/PowerManager#getBatteryDischargePrediction()

[5] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau *et al.*, "MLPerf Tiny benchmark," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. [Online]. Available: https://arxiv.org/abs/2106.07597

[6] F. E. Harrell, R. M. Califf, D. B. Pryor, K. L. Lee, and R. A. Rosati, "Evaluating the yield of medical tests," *JAMA*, vol. 247, no. 18, pp. 2543–2546, 1982.

[7] S. Heydari and Q. H. Mahmoud, "Tiny machine learning and on-device inference: A survey of applications, challenges, and future directions," *Sensors*, vol. 25, no. 10, p. 3191, 2025.

[8] N. N. Alajlan and D. M. Ibrahim, "TinyML: Enabling of inference deep learning models on ultra-low-power IoT edge devices for AI applications," *Micromachines*, vol. 13, no. 6, p. 851, 2022.

[9] S. Albelali and M. Ahmed, "Hidden leaks in time series forecasting: How data leakage affects LSTM evaluation across configurations and validation strategies," *arXiv preprint arXiv:2512.06932*, 2025. [Online]. Available: https://arxiv.org/abs/2512.06932

[10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.